Østfold University College

User Guide TaskyFy(2.0.0)

ITF20123-1 25V Rammeverk og .NET

Andreas Isaksen
Arshad Abba
Kjell-Magne Larsen
Marius Rørmark
Hedda Ørnelund Nilsen

May 27, 2025
Halden

Contents

Introduction

1.1 What's a Scheduler?

A person or device that determines a schedule, that determines the order that tasks are to be done(Wordink, scheduler).
This means that a series of task can be initialized based on different trigger systems. The triggers can vary from time-based actions to flag actions.

1.2 What this Library contains.

This library contains the source code for the first release of TaskyFY. It has the application source code, a CLI and unit tests for different classes. TaskyFy contains classes, abstract classes, interfaces, handlers, and time/event based task execution. For the user importing the NuGet library, there are builders which simplifies the process of creating tasks.

1.3 How does this Scheduler work?

Scheduler contains functioning solutions for.

- **Timer**
  Time based triggers is used to make different types of time based action. With this it would be possible to schedule task to initialise on specific dates and times.
  This would also open the possibility to have a framework that takes special days into consideration.
- **Iteration**
  Functionality that lets the user make a task into a repetitive action based on determined specifications.
  This would open the possibility of creating a single task that can be repeated based on the given criteria.
  The given criteria should be flexible, so that the triggers can be defined on time-based criteria, action based triggers, or a combination of both.
- **Actions**
  Action based triggers let's the used created custom specifications that would do certain taks when the trigges gets flagged.
- **Return Values**
  With the new implementation of `Func<T>` delegates, it is now possible to get return values from created jobs.

Migrating from 1.0.0 to 2.0.0

2.1 Critical Changes

To make **Advanced Jobs** functional, which use `Func<T> delegations` to make it possible to creates jobs with return values. There had to be made some minor critical changes to the fundamental structure regarding to how jobs are created.
Much of the *Job* class has been abstracted up to its parent class *JobDefinition* which now is the class used to build new jobs.
All other other structure implementation is still the same, only with more features to choose from. This means that all the jobs that has been created using **1.0.0** will still be functional and work as intended when the class definition of your *Job* object gets renamed to *JobDefinition*(or just a *var* object). The code below this section illustrate the syntactical difference between created jobs in **1.0.0** and **2.0.0**, and further information about this can be found in chapter 5 Fundamental Usage.

1.0.0	**2.0.0**

`// Create a job using the builder.`
`Job job = new JobBuilder()`
`  //Logic for building job`
	`// Create a job using the builder.`
	`JobDefinition job = new JobBuilder()`
	`  //Logic for building job`

Table 2.1: Syntactical difference between 1.0.0 and 2.0.0

2.2 New Features

2.2.1 Advanced Jobs

Advanced jobs are a new way to create and implement jobs in our framework, that allows the user to create jobs with `Func<T>` delegates. This allows users to create action job with the possibility of getting return values on the job object, rather than just preforming *void* actions. Further information about how to use this solution can be found in chapter 5 Fundamental Usage and 6 API-References.

2.2.2 Logging

This project has a custom logging system that writes log messages to a file. It uses a class called `SimpleFileLogger`, which follows the `ILogger` interface from Microsoft's logging system.
The idea is to have a basic and easy way to keep track of what's going on in the app, especially for debugging or checking how things run over time.

When the application starts, it creates a new log file inside a folder called "Logs".The file name includes the current date and time, so a new file is made every time the app runs.

All log messages are written to this file with a timestamp, the log level (like Information, Warning, or Error), and the name of the class that logged the message. If there's an exception, the details are also added to the log so it's easier to find out what went wrong.

There's also a static class called `Logging` that helps you create loggers.
You can make a logger by passing in a category name or just the type of the class. This makes it simple to use logging in different parts of the app without repeating the setup code.

For example, in the `JobHandler` class, we use a logger to write a message when a job starts, like this:

```
1 private static readonly ILogger logger = Logging.CreateLogger<JobHandler>()
      ;
2
3 logger.LogInformation("Job started with ID: " + jobStorageId);
```

Listing 2.1: Implementing logging Example

This kind of logging helps us follow what the app is doing, find problems, and understand the flow of events, especially when working on or fixing the code later.

2.2.3 Retry Logic

2.2.3.1 JobDefinition – Retry Functionality

A simple automatic retry mechanism has been implemented in `JobDefinition.ExecuteJobAsync()` to handle transient errors during job execution.
This allows jobs to retry without manual intervention.

2.2.3.2 Configuration

- **MaxRetries** (property on `JobDefinition`)
  Determines the maximum number of additional attempts after an initial failure. The default value is 0 (no retries), which preserves the behavior from Version *1.0.0*.
  Set this to > 0 to enable retries.
- **RetryDelay** (property on `JobDefinition`)
  Defines the time between each retry attempt. The default is 2 seconds.

2.2.3.3 Error Handling

- **IsExceptionTransient(Exception ex)** (virtual method in JobDefinition)
  A simple check to decide whether an error is transient and should trigger a retry.
  The default implementation treats most errors as transient, except for *OperationCanceledException*,

NullReferenceException, and *ArgumentException*.
Subclasses can override this for more specific logic.

- If a job fails all configured attempts (the initial attempt plus *MaxRetries*), or a non-transient error occurs, **ExecuteJobAsync** will throw a new *NumberOfRetriesException*.
  This exception contains the last original error as its *InnerException*, as well as the configured *MaxRetries* value and **AttemptsMade** (the total number of attempts performed).

2.2.4 JobHandler

2.2.4.1 New Methods

Int StartOrAddJob(JobDefinition job):
Searches through existing jobs using the reference object instead of storageId. This method essentially combines AddJob and StartJob. It's more flexible and requires less code for the user.
It has two possible outcomes:

- Job exists: Starts it and returns the existing storageId.
- Job does not exist: Adds the job and returns its new storageId.

Void StopJob(JobDefinition job):
Overload that allows stopping a job by reference object.

Void RemoveJob(JobDefinition job):
Overload that allows removing an existing job by reference object.

2.2.4.2 Updated Job Lifecycle

The **CycleJobAsync** method in *JobHandler* now calls `job.ExecuteJobAsync()`, which includes the newly implemented retry logic(see section 2.2.3 Retry Logic)

CycleJobAsync now specifically catches *NumberOfRetriesException* (in addition to *OperationCanceledException* and general *Exception*) thrown by `job.ExecuteJobAsync()`. This is done to correctly set the **JobStatus** to *Failed* and log the error (including details from *NumberOfRetriesException* such as *InnerException*, *AttemptsMade*, and *MaxRetries*) via the existing **ILogger**.

2.3 Other changes

2.3.1 Console Feedback to Logg

Some feedback such ass start-time, intervall, and end-time of started jobs that previously was shown in the console window, is now being sent to logg files. This was done with the intention of building a solution that gives a cleaner console window and more organised logging of technical information.

2.3.2 JobDefinition

Description property:
If set to null or an empty string, the placeholder “No description provided” will be used instead of throwing an ArgumentNullException. This is primarily to make SetDescription in JobDefinitionBuilder optional.

2.3.3 JobDefinitionBuilder

SetScheduled:
This method has been marked obsolete with the [Obsolete] attribute.

2.4 Bug Fixes

- **Event trigger:** Executes the correct action when it is based on another job in the job storage.

Installation

3.1 Summary

This is a short introduction on how to install the scheduler library framework on your local *Visual Studio 2022* EDI.

3.2 NuGet Package

To install the NuGet package into Visual Studio to use in a new project. You'll need a local package directory on your client.
If you don't have this already, then you can easy create one with this via command line like this;

```
1 mkdir C:$\backslash$LocalNuGetFeed
```

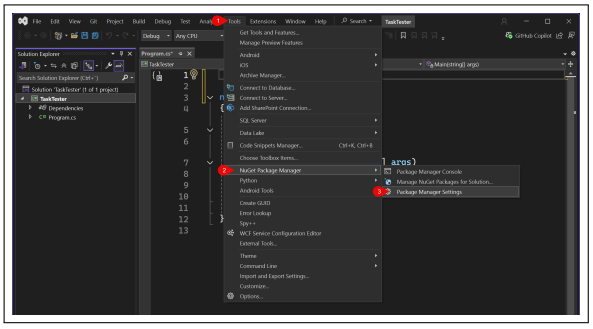
Listing 3.1: Creates a package folder in C:

Move the file *Hiof.TaskyFy.2.0.0.nupkg* to this folder (or the folder of your choosing).

3.3 Install NuGet To Visual Studio

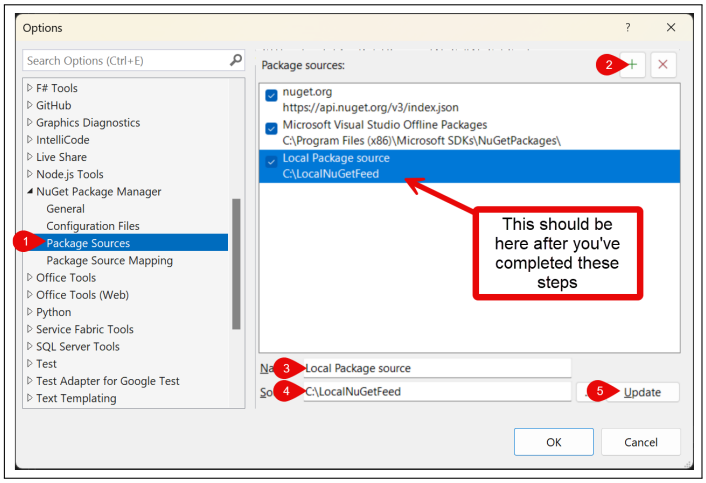
Now that you have the NuGet file stored on your local computer, you can install it in your Visual Studio 2022 EDI with the following steps.

1. Open your project in Visual Studio.
2. When the project is open, go to:
    1. Tools (in taskbar)
    2. NuGet Package Manager
    3. Package Manager Settings



3. You should now have a *Options* window on your screen.
   Do the following steps to install the local NuGet package into Visual Studio;

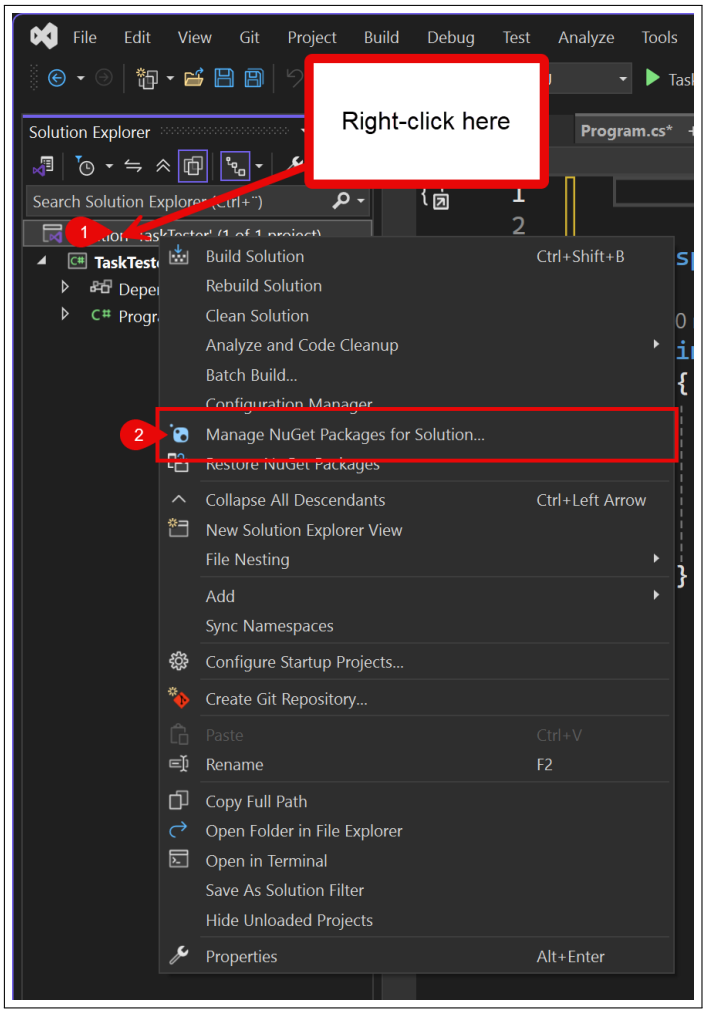
1. Navigate in the sidebar to *Package Sources*
2. Press the '+' sign in the top right corner.
3. Give a name to your local NuGet source (e.g. Local Package source).
4. Enter the location where your NuGet package is stored. If you followed the instructions on creating a location for your NuGet above, then this should be C:\LocalNuGetFeed
5. Press det *'update'* button.



3.4 Adding NuGet To Project

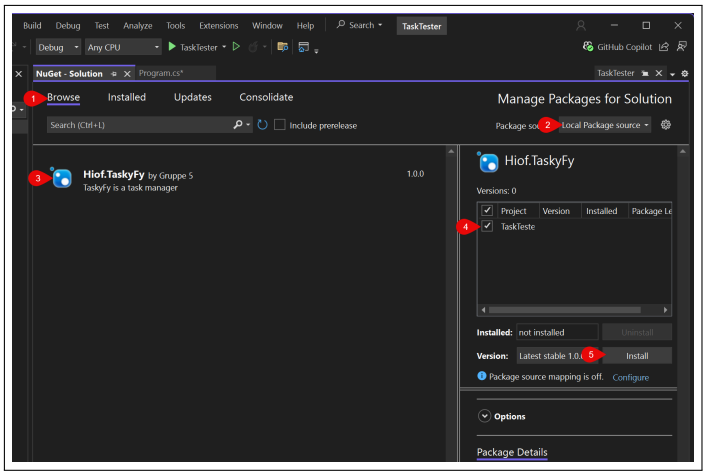
Now that the NuGet package is added to available packages in Visual Studio, you should be able to add it to your project with the following steps.

1. Right-click on your project solution to get the context menu.
2. Click on *Manage NuGet Packages for Solution...*.



This should open a new windows in Visual Studio called *'NuGet - Solution'*. Here you can follow these steps to install the NuGet package into your project.

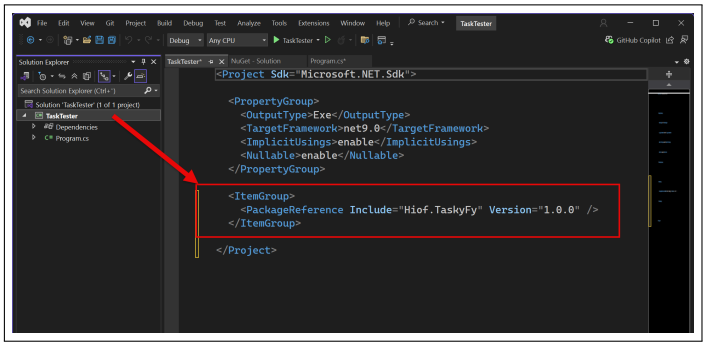
1. Click on *'Browse'* in the top part of the window.
2. Choose the 'Local Package Source' (Or the name you gave your local source) from the *Package source* list.
3. Click on the NuGetPackage called *Hiof.TaskyFy* in the list.
4. Check of the project in the list(on the right) you wish to install the package in.
5. Press the *Install* button under the checklist from last step.
6. Press the *'Apply'* button in the *'Preview Changes'* window that pops up after you've pressed on the install button.



The NuGet package should now be fully accessible to your project.

3.5 Control Project Access

If you're still experiencing issues regarding access to the NuGet package. You can control that the NuGet has been added in the given class XML. You can control this by clicking on the project and make sure the project XML contains this line as shown in the image and control that the correct version is implemented.



General Structure

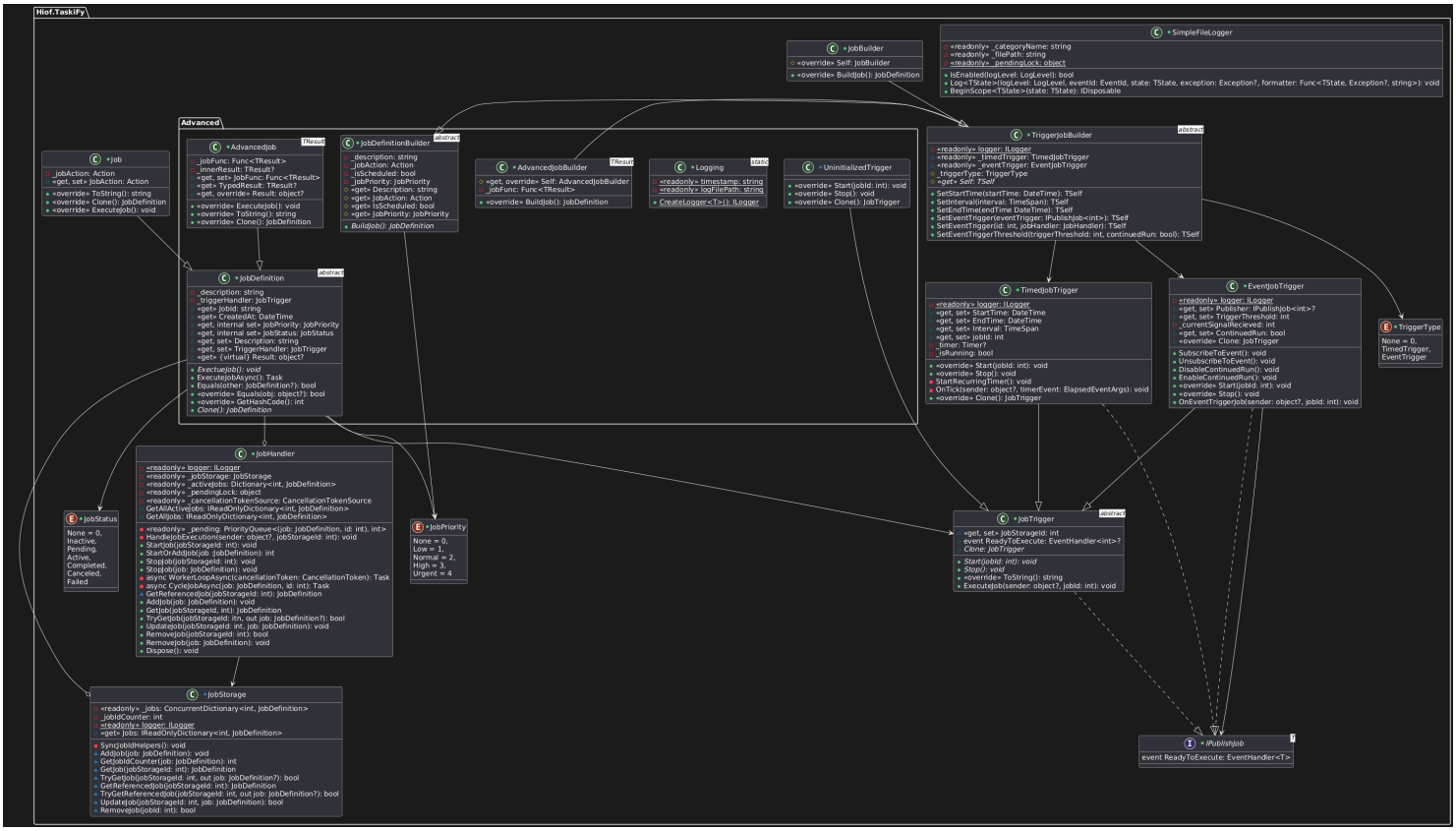


Figure 4.1: UML diagram of framework

Fundamental Usage

> ***IMPORTANT***
>
> **V2.0.0** uses the class `JobDefinition` to create jobs, unlike **V1.0.0**, which used the `Job` class.
> More info in chapter *2.Migrating from 1.0.0 to 2.0.0*.

5.1 Introduction

This chapter introduces the core concepts and typical workflows when using the scheduler framework. It's designed to help you quickly understand how to define, schedule, and manage jobs using the provided API (see Chapter 6 API-References).

5.2 Overview

At its core, the framework allows you to schedule tasks, which is refereed to as *'jobs'*.
Jobs can execute at specific times or when a certain event occur.
The system is built around a few fundamental components:

- **Job Definitions**
  Represents the tasks to be executed.
- **Job Triggers**
  Control when a job is executed (time-based or event-based).
- **Job Builders**
  Provide a fluent interface for creating job interfaces.
- **Job Handling**
  Manages the lifecycle and storage of jobs.

5.2.1 Defining a Job

5.2.1.1 One-Time Job

A job is defined using the **JobBuilder** class to construct an instance of the **JobDefinition** class. When creating a job, you specify a start time, a description, and a lambda expression (an **Action**) that encapsulates the code to execute. You can also mark the job as scheduled. This approach follows the builder pattern for flexible job configuration.

```
1 // One-Time Job Example: Schedules a data synchronization job to run 5
      minutes from now.
```

```
JobHandler jobHandler = new JobHandler();
JobDefinition oneTimeJob = new JobBuilder()
    .SetStartTime(DateTime.Now.AddMinutes(5))
    .SetDescription("One-time data synchronization")
    .SetJobAction(() =>
    {
        // Code to synchronize data
        Console.WriteLine("Synchronizing data...");
    })
    .BuildJob();
```

Listing 5.1: Example Usage of JobBuilder

5.2.1.2 Recurring Job

A recurring job is also created using the **JobBuilder** class. In addition to setting a start time, description, and action, you specify an interval to indicate how often the job should repeat. This is useful for tasks that need to run on a fixed schedule, such as system monitoring or maintenance routines.

```
// Recurring Job Example: Schedules a system health check to start in 1
    hour and recur every hour.
JobHandler jobHandler = new JobHandler();

JobDefinition recurringJob = new JobBuilder()
    .SetStartTime(DateTime.Now.AddHours(1))
    .SetInterval(TimeSpan.FromHours(1))
    .SetDescription("Hourly system health check")
    .SetJobAction(() =>
    {
        // Code to perform a system health check
        Console.WriteLine("Performing system health check...");
    })
    .BuildJob();
```

Listing 5.2: Recurring Job Example

5.2.1.3 Event-Driven Job

An event-driven job is triggered by the occurrence of a specific event rather than a fixed time or interval. Using the **JobBuilder** and **JobHandler** classes, you can configure the job to respond to another job's execution or any predefined event. In this example, the job is set to trigger when a job with ID 2 is executed, such as after a new user registration. A threshold can also be set to control how often the job should respond to the event.

```
// Event Job Example: schedules a Welcome email to be sent every time a new
    registration is executed (job already existing with id 2)
JobHandler jobHandler = new JobHandler();

JobDefinition eventJob = new JobBuilder()
    .SetEventTrigger(2, jobHandler)
    .SetEventTriggerThreshold(1, true)
```

```
    .SetDescription("Send welcome email on user registration")
    .SetJobAction(() =>
    {
        // Code to send a welcome email.
        Console.WriteLine("Sending welcome email...");
    })
    .BuildJob();
```

Listing 5.3: Event-Driven Job Example

5.2.1.4 Event-Driven Job with IPublishJob<T> and Custom Event Publisher Implementation

If an external class is implemented with `IPublishJob<int>`, one can connect a *job* directly to the given method call by passing the trigger object to `SetEventTrigger(object.TriggerMethod)`. This will create a *EventJobTrigger* that subscribes to the method's *ReadyToExcecute* event and fires the defined *JobAction* when the trigger call is raised.

```
// And external class implementing IPublishJob<int>
internal class ExampleClass : IPublishJob<int>
{
    public event EventHandler<int>? ReadyToExecute;

    public void TriggerMethod(int jobId)
    {
        ReadyToExecute?.Invoke(this, jobId);
    }
}

// Fully implemented solution for a triggered event with a publisher.
static void Main(string[] args)
{
    // Create a JobHandler object.
    JobHandler jobHandler = new();

    // Creates a publisher object from the example class.
    ExampleClass ExampleObject = new ExampleClass();

    // Create a job using the builder.
    JobDefinition job1 = new JobBuilder()
        .SetEventTrigger(ExampleObject)
        .SetEventTriggerThreshold(1, true)
        .SetDescription("Simluates trigger by method call")
        .SetJobAction(() =>
        {
            // Code to execute on trigger call.
            Console.WriteLine("TriggerMethod has been called");
        })
        .BuildJob();

    // Add built job to jobHandler.
    jobHandler.AddJob(job1);

    // Start the job using the ID generated by jobHandler.
    jobHandler.StartJob(1);

```

```
    Thread.Sleep(1000);

    // Calls triggermethod to execute job.
    ExampleObject.TriggerMethod(1);

    // Halt the program from ending.
    Console.ReadLine();
}
```

Listing 5.4: Event-Driven Job with IPublishJob<T> Example

5.2.1.5 AdvancedJob<TResult> – Recurring Job

Advanced recurring jobs are similar to a regular recurring job, but with some syntactical differences, and the ability to **return** values. This solution is useful for tasks that needs to retrieve data on a set intervall.
Here the *JobAction* is defined by using **jobFunc** to implement the desired code functionality for the job by using a **lambda** expression, and the job is run on the defined time intervall.

```
  // Create a TimedJobTrigger object with time properties.
  TimedJobTrigger timeTrigger = new()
  {
    StartTime = DateTime.Now.AddSeconds(5),
    Interval = TimeSpan.FromSeconds(10)
  };

  // Builds a job that fetches data from a CSV file.
  JobDefinition fetchJob = new AdvancedJob<string>(
    description: "Web string",
    triggerHandler: timeTrigger,
    jobFunc: () => new HttpClient()
          .GetStringAsync("http://www.it.hiof.no/statok/katters_vekt.csv")
          .GetAwaiter().GetResult()
  );

  handler.AddJob(fetchJob);

  // Grab the JobId
  var storageId = handler.GetAllJobs
            .First(kvp => kvp.Value.JobId == fetchJob.JobId)
            .Key;
  handler.StartJob(storageId);

  // Sleep threads are added to make sure the job is completes before
  // the program tries to return the collected data.
  Thread.Sleep(15_000);

  Console.WriteLine("Fetched payload:");
  Console.WriteLine(fetchJob.Result);

  Console.ReadLine();
```

Listing 5.5: AdvancedJob<TResult> – Recurring Job Example

5.2.1.6 AdvancedJob<TResult> – Event-Driven Job

Advanced event driven jobs are similar to regular event driven jobs, but with some syntactical differences, and the ability to **return** values. This solution is useful for tasks that needs to retrieve data When a certain event is triggered.
Here the *JobAction* is defined by using **jobFunc** to implement the desired code functionality for the job by using a **lambda** expression, and the job is run on the defined event trigger.

```
// ExampleClass is defined to illustrate a dummy event.
internal class ExampleClass : IPublishJob<int>
{
  public event EventHandler<int>? ReadyToExecute;

  public void TriggerMethod(int jobId)
  {
    ReadyToExecute?.Invoke(this, jobId);
  }
}

// Object from dummy class is created to access trigger event.
ExampleClass triggerObject = new ExampleClass();

// Trigger object is defined.
EventJobTrigger eventTrigger = new(triggerObject);

// Builds a job that fetches data from a CSV file.
JobDefinition fetchJob = new AdvancedJob<string>(
  description: "Web string",
  triggerHandler: eventTrigger,
  jobFunc: () => new HttpClient()
    .GetStringAsync("http://www.it.hiof.no/statok/katters_vekt.csv")
    .GetAwaiter().GetResult()
);

handler.AddJob(fetchJob);

// Grab the JobId
var storageId = handler.GetAllJobs
          .First(kvp => kvp.Value.JobId == fetchJob.JobId)
          .Key;
handler.StartJob(storageId);

// Sleep thread makes sure the job has started before triggering event.
Thread.Sleep(10_000);
// Trigger event.
triggerObject.TriggerMethod(storageId);
// Sleep thread makes sure the job has completed before retrieving data.
Thread.Sleep(10_000);
Console.WriteLine("Fetched payload:");
Console.WriteLine(fetchJob.Result);

Console.ReadLine();
```

Listing 5.6: AdvancedJob<TResult> – Event-Driven Job Example

5.2.2 Scheduling with Triggers

Triggers are responsible for determining the execution criteria of the job:

- **TimedJobTrigger**:
  Use this trigger when you want to schedule a job based on specific time criteria.
  You can set:
  - **StartTime**: The moment when the job should first run.
  - **Interval**: The recurring time span between executions (optional).
  - **EndTime**: When the job should stop running (optional).
    This trigger automatically handles the timing logic, initiating the job execution either immediately (if the start time has passed) or after a calculated delay.
- **EventJobTrigger**:
  For event-driven scenarios, the `EventJobTrigger` listens to signals from a publisher. It can be configured to execute the job once or repeatedly based on a threshold of event signals.

A placeholder trigger, `UninizializedTrigger`, exists to enforce that a valid trigger is always set before a job i scheduled.

5.2.3 Building Jobs Using Fluent Builders

To simplify the process of configuring jobs, the framework offers a fluent API via the **JobBuilder** class. This builder allows you to chains configuration methods to set up:

- **Description**: A short, meaningful description of the job.
- **Trigger Details**:
  - For time-based scheduling: `SetStartTime()`, `SetInterval()`, and `SetEndTime()`.
  - For event-driven scheduling: `SetEventTrigger()` and `SetEventTriggerThreshold()`.
- **Job Action**: The task logic defined as an **Action**.
- **Job Priority**: Set the importance of the job.

Once all parameters are defined, calling `BuildJob()` returns a fully configured job instance ready for scheduling.

5.2.4 Handling Jobs

JobHandler

The **JobHandler** class is the central component for managing the lifecycle of jobs.
Its responsible for the following:

- **Adding Jobs**: Use `AddJob()` to store new job definitions.
- **Starting Jobs**: When `StartJob()` is called, the handler subscribes to the job's trigger and marks it a active. When the trigger fires, the job is executed asynchronously.
- **Managing Jobs**:
  - Updating Job statuses (e.g., from Pending to Active to Completed).
  - Removing jobs using `RemoveJob()`
- **Monitoring Active Jobs**: Methods like `ReadActiveList()` provide a quick summary of currently active tasks.

Internally, the **JobHandler** works with **JobStorage** component, which is responsible for persisting job definitions and ensuring that each job has a unique storage ID.

5.2.5 Putting It All Together

A typical workflow using the framework follows these steps.

1. **Define the Job**:
   Create a job through the `JobBuilder`(recommended for more fluent setup), or directly in the Job class.
2. **Configure the Trigger**:
   Choose a trigger based on the requirements:
   - Use `TimedJobTrigger` for time-based scheduling.
   - Use `EventJobTrigger` for event-based scheduling.
3. **Add the Job to the Handler**:
   Use the `JobHandler.AddJob()` method to register the job within the system.
4. **Get the Job ID**
   Before starting the job, you'll have to get the generated ID for the current job you've added to the handler. This can be retrieved in several ways.
   - Get the first job stored in queue:
     `int jobId = jobHandler.GetAllJobs.Keys.First();`
   - Generate a list of all created jobs stored in **JobStorage** which can be iterated to get a list of jobs (in this suggested solution `.GetAllJobs` can be replaced with `.GetAllActiveJobs` to only list active jobs in queue):

```
IReadOnlyDictionary<int, JobDefinition> all = handler.GetAllJobs;
  foreach (var job in all)
  {
    int id      = job.Key;
    var def     = job.Value;
    Console.WriteLine(
      $"ID = {id}, " +
      $"Description = "{def.Description}", " +
```

```
    }

```

Listing 5.7: Print a list of created jobs

5. **Start the Job**:
   Call `JobHandler.StartJob(jobId)` to subscribe the job to its trigger and start monitoring for execution.

6. **Monitor and Manage**:
   As job execute, the framework updates their status.

API-References

6.1 Core Classes

Class	Type	Constructors	Properties/Events

JobDefinition	AdvancedJob<TResult>	`AdvancedJob<TResult>(`
		`string description,`
		`JobTrigger triggerHandler,`
		`Func<TResult> jobFunc);`
		
		`AdvancedJob<TResult>(`
		`string description,`
		`JobTrigger triggerHandler,`
		`Func<TResult> jobFunc,`
		`JobPriority jobPriority);`
			Func<TResult> JobFunc { get; }
			
			TResult? TypedResult { get; }
			
			override object? Result { get; }

JobDefinition	Job	`Job( string description,`
		`JobTrigger triggerHandler,`
		`Action jobAction)`
		
		`Job( string description,`
		`JobTrigger triggerHandler,`
		`Action jobAction,`
		`JobPriority jobPriority)`
			JobAction

EventJobTrigger	IPublishJob<int>	`EventJobTrigger(`
		`IPublishJob<int> eventPublisher);`
		
		`EventJobTrigger(`
		`IPublishJob<int> eventPublisher,`
		`int triggerThreshold = 1,`
		`bool continuedRun = false);`
			Event

6.2 Job Interfaces & Enumerations

Interface	Members
IPublishJob`<T>`	Event: `ReadyToExecute`

Enumeration	Values
JobPriority	None Low Normal High Urgent
JobStatus	None Inactive Pending Active Completed Canceled Failed
TriggerType	None TimedTrigger EventTrigger

6.3 Job Builders

Builder	Type	Methods

JobDefinitionBuilder	Abstract Base Class	`SetDescription(string description)`
		`SetJobAction(Action jobAction)`
		`SetScheduled(bool isScheduled)`
		`SetJobPriority(JobPriority jobPriority)`
		`BuildJob()` (abstract)

JobBuilder	Concrete Implementation	**Timed:**
		`SetStartTime(DateTime startTime)`
		`SetInterval(TimeSpan interval)`
		`SetEndTime(DateTime endTime)`
		Event:
		`SetEventTrigger(IPublishJob<int> eventTrigger)`
		`SetEventTrigger(int id, JobHandler jobHandler)`
		`SetEventTriggerThreshold(int triggerThreshold, bool continuedRun)`
		Finalize:
		`BuildJob()`

6.4 Job Triggers

Trigger	Methods	Properties

JobTrigger	`Start(int jobId)`
	`Stop()`
	`Clone()`
		`ExecuteJob(object? sender, int jobId)`

TimedJobTrigger	`Start(int jobId)`
	`Stop()`
	`Clone()`
		`DateTime StartTime } get; set; }`
		`DateTime EndTime { get; set; }`
		`TimeSpan Interval { get; set; }`
		`int JobId { get; set; }`

EventJobTrigger	`SubscribeToEvent()`
	`UnsubscribeToEvent()`
	`EnableContinuedRun()`
	`DisableContinuedRun()`
	`Start(int jobId)`
	`Stop()`
	`OnEventTriggerJob(object? sender, int jobId)`
	`Clone()`
		`IPublishJob<int>? Publisher { get; set; }`
		`int TriggerThreshold { get; set; }`
		`bool ContinuedRun { get; set; }`

6.5 Handling & Storage

Component	Commands	Queries	Properties

JobHandler	`StartJob(int jobId)`
	`StopJob(int jobId)`
	`AddJob(JobDefinition)`
	`UpdateJob(int, JobDefinition)`
	`RemoveJob(int)`
		`GetJob(int) → JobDefinition`
		`TryGetJob(int, out JobDefinition) → bool`
		`StartOrAddJob(JobDefinition) → int`
		`StopJob(JobDefinition)`
		`IReadOnlyDictionary<int, JobDefinition>`
		`RemoveJob(JobDefinition)`
			GetAllJobs
			GetAllActiveJobs

JobStorage	`AddJob(JobDefinition)`
	`UpdateJob(int, JobDefinition)`
	`RemoveJob(int)`
		`GetJob(int) → JobDefinition`
		`TryGetJob(int, out JobDefinition) → bool`
			Jobs (readonly)

Bibliography

[1] Christian Nagel. *Professional C# and .NET*. 2021st ed., Wrox, 2021. ISBN: 978-1-119-79720-3.

[2] Krzysztof Cwalina and Brad Abrams. *Framework Design Guidelines: Conventions, Idioms, and Patterns for Reusable .NET Libraries*. 3rd ed., Addison-Wesley Professional, 2019. ISBN: 978-0-13-589646-8.

[3] Microsoft Docs. *Events in .NET*. Documentation on events in .NET, available at: https://learn.microsoft.com/en-us/dotnet/standard/events/.

[4] Microsoft Docs. *Async/Await - Best Practices in Asynchronous Programming*. Archived article from MSDN Magazine, March 2013, available at: https://learn.microsoft.com/en-us/archive/msdn-magazine/2013/march/async-await-best-practices-in-asynchronous-programming.

[5] Microsoft Docs. *Choosing Between DateTime, DateTimeOffset, TimeSpan, and TimeZoneInfo*. Guidance on handling date and time in .NET, available at: https://learn.microsoft.com/en-us/dotnet/standard/datetime/choosing-between-datetime.

[6] Microsoft Docs. *Best Practices for Exceptions*. Guidelines on exception handling in .NET, available at: https://learn.microsoft.com/en-us/dotnet/standard/exceptions/best-practices-for-exceptions.

[7] OpenAI. *ChatGPT analysing framework*. Personal communication, March 31, 2025. Available at: https://chatgpt.com/share/67eaf428-2478-800e-a0d0-6942c3410a8f

[8] OpenAI. *ChatGPT Conversation: Framework API Analysing*. Accessed May 27, 2025. Available at: https://chatgpt.com/share/68358d66-aa6c-800e-a00a-86fccaef3979.

[9] Microsoft Docs. *Timer Class*. How to implement Timer Class in .NET. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.timers.timer?view=net-9.0

[10] Stack Overflow. *Cleanest way to write retry logic?* Accessed May 27, 2025. Available at: https://stackoverflow.com/questions/1563191/cleanest-way-to-write-retry-logic

[11] Code Maze. *How to Implement Retry Logic in C#*. Accessed May 27, 2025. Available at: https://code-maze.com/csharp-implement-retry-logic/

[12] Microsoft Learn. *Retry pattern - Azure Architecture Center*. Accessed May 27, 2025. Available at: https://learn.microsoft.com/en-us/azure/architecture/patterns/retry

[13] Microsoft Learn. *Configurable retry logic in SqlClient introduction*. Accessed May 27, 2025. Available at: https://learn.microsoft.com/en-us/sql/connect/ado-net/configurable-retry-logic-sqlclient-introduction?view=sql-server-ver17

[14] Microsoft Learn. *Using Delegates (C# Programming Guide)*. Accessed May 27, 2025. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/using-delegates

[15] Microsoft Learn. *Delegates (C# Programming Guide)*. Accessed May 27, 2025. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/

[16] Microsoft Q&A. *Logging in C# - To a text file*. Accessed May 27, 2025. Available at: https://learn.microsoft.com/en-us/answers/questions/1377949/logging-in-c-to-a-text-file